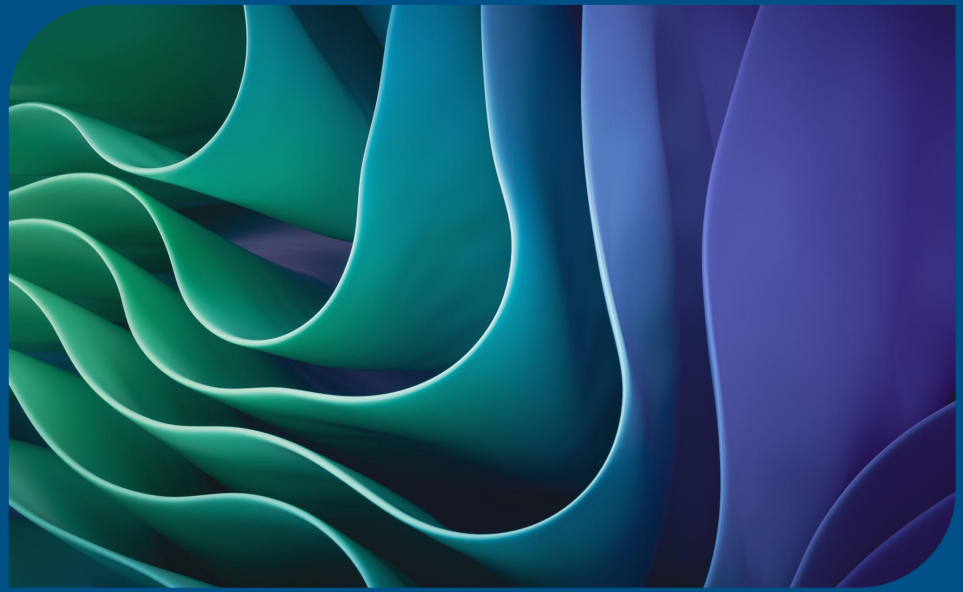
Predicting Patient Insurance Costs: A Regression Analysis of Demographics and Health Factors

Everett Ypsilantis & Ananya Jhamb



Assignment Introduction:

- Medical insurance pricing must balance affordability for individuals with financial sustainability for insurers. Our project uses a real-world dataset of patient demographics and health factors to model insurance charges.

- We developed a regression model to predict these charges as accurately as possible.

- Model performance was evaluated using mean squared error (MSE), the project's required metric.

The Data

- age: age (in years)
- sex: gender (male = 0, female = 1)
- bmi: body mass index
- children: the number of offspring of the individual
- smoker: smoking status (no = 0, yes = 1)
- region: geo-location (northwest = 0, northeast = 1, southeast = 2, southwest = 3)
- charges: cost of medical insurance [target]

```
# Inspect columns and confirm target
df.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

```
#Check column data types
df.dtypes
```

	0
age	int64
sex	object
bmi	float64
children	int64
smoker	object
region	object
charges	float64

dtype: object

Read in Training Data

```python
# Read in training data

import pandas as pd

# Replace with your Google Drive shareable link
drive_url = "https://drive.google.com/file/d/122kxiprQhzR123mGnyGxSERg854FmbBF/view?usp=sharing"

# Convert the Google Drive URL to a direct download URL
file_id = drive_url.split('/d/')[1].split('/')[0]
download_url = f"https://drive.google.com/uc?id={file_id}"

# Read the CSV file into a DataFrame
df = pd.read_csv(download_url)

# Display the first few rows of the DataFrame
print(df.head())
```

```
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520
```

Read in Test Data

```
# Read in test data

import pandas as pd

# Replace with your Google Drive shareable link
drive_url = "https://drive.google.com/file/d/1qZIgsTvpF4acFXosreetuGZyYaOjhSgM/view?usp=sharing"
# Convert the Google Drive URL to a direct download URL
file_id = drive_url.split('/d/')[1].split('/')[0]
download_url = f"https://drive.google.com/uc?id={file_id}"

# Read the CSV file into a DataFrame
df_test = pd.read_csv(download_url)

# Display the first few rows of the DataFrame
print(df_test.head())
```

```
   age  sex        bmi  children  smoker  region
0   58    0  30.338722         0       0       1
1   60    0  24.324606         0       0       0
2   27    0  18.228846         3       0       1
3   24    0  40.426077         0       1       2
4   19    0  35.872515         0       0       0
```

Linear Regression

Encode Categorical Variables

```
#Objects mast be converted to numbers
#Encode Categorical Variables

from sklearn.preprocessing import LabelEncoder

#Copy df so we don't overwrite original
df_encode = df.copy()
df_test_encode = df_test.copy()

#Columns that need encoding
cat_cols = ['sex','smoker','region']

le = LabelEncoder()

for col in cat_cols:
  df_encode[col] = le.fit_transform(df_encode[col])
  df_test_encode[col] = le.fit_transform(df_test_encode[col])
```

```
#Redefine X and y using the encoded data
X = df_encode.drop('charges', axis=1)
y = df_encode['charges']
```

Regression Model

MSE = 33,635,210.43

```
#Split, train, and evaluate
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X_train, X_vale, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

y_val_pred = lin_reg.predict(X_vale)

mse = mean_squared_error(y_val, y_val_pred)
print(f"Mean Squared Error: {mse}")
```

```
Mean Squared Error: 33635210.431178406
```

Random Forest Regressor

Preparing the Random Forest Regressor

```python
#Re-run your train/validation split
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```python
#Improve Score with Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor

# Create a Random Forest Regressor
rf = RandomForestRegressor(
    n_estimators=300,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```python
#Fit the model
rf.fit(X_train, y_train)
```

```
RandomForestRegressor
```

Random Forest Regressor

MSE - 20,778,721.78

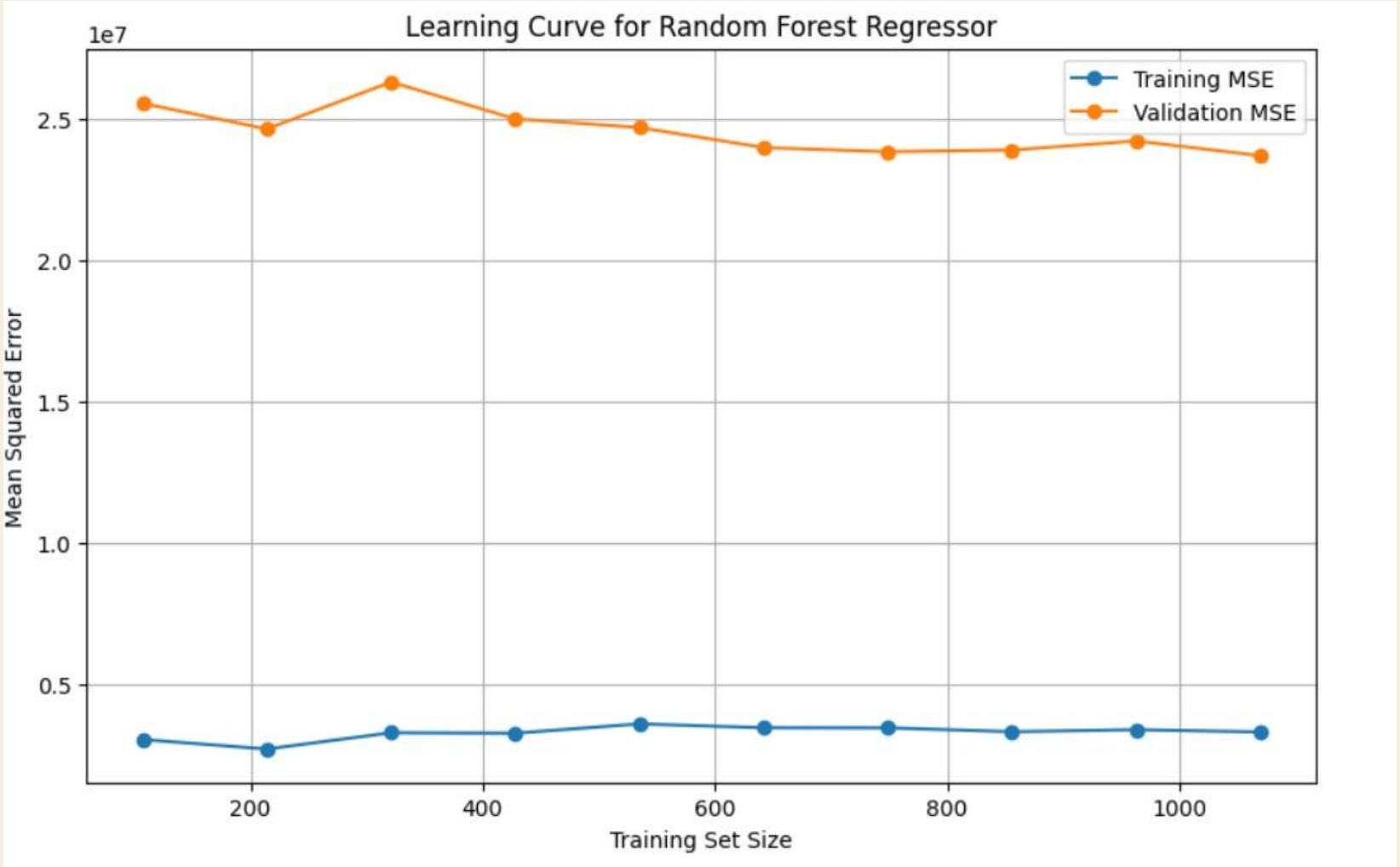
```
#Validate using MSE
from sklearn.metrics import mean_squared_error

y_val_pred = rf.predict(X_val)
mse = mean_squared_error(y_val, y_val_pred)
print(f"Mean Squared Error: {mse}")
```

```
Mean Squared Error: 20778721.787278876
```

Learning Curve Visualization

Learning Curve Script

```python
#Learning Curve Visualization
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt
import numpy as np

#Generate learning curve data
train_sizes, train_scores, val_scores = learning_curve(
    estimator=rf,
    X=X,
    y=y,
    cv=5,
    scoring='neg_mean_squared_error',
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)

#Convert negative MSE to positive

train_mse = -train_scores.mean(axis=1)
val_mse = -val_scores.mean(axis=1)

#Plot the learning curve

plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mse, marker='o', label='Training MSE')
plt.plot(train_sizes, val_mse, marker='o', label='Validation MSE')

plt.title('Learning Curve for Random Forest Regressor')
plt.xlabel('Training Set Size')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.grid(True)
plt.show()
```


Learning Curve for Random Forest Regressor
1e7
Training MSE
Validation MSE
Mean Squared Error
2.5
2.0
1.5
1.0
0.5
200
400
600
800
1000
Training Set Size

Retrain on full Training Data

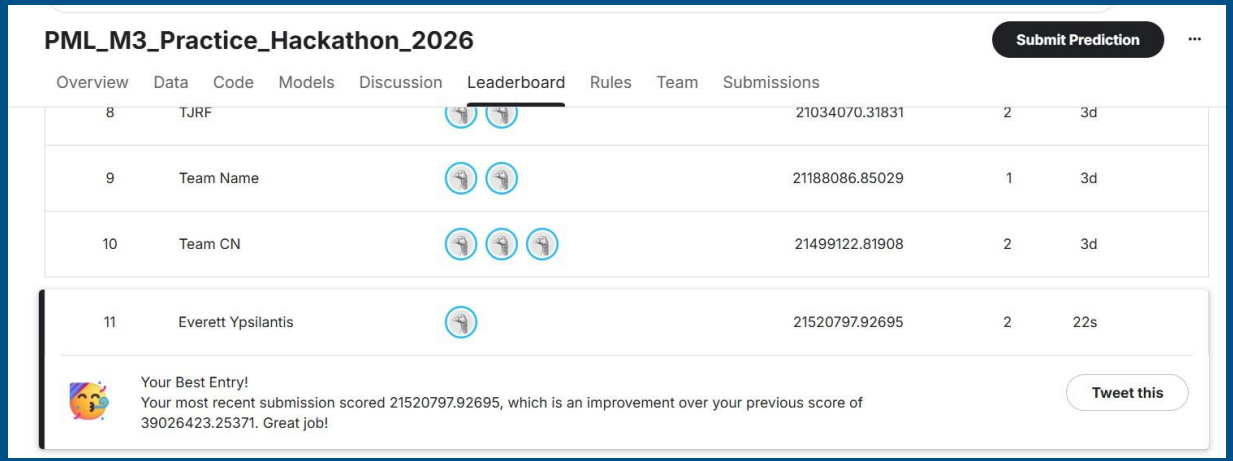
Predict on Test Set

```
#Retrain on full training data
rf.fit(X, y)
```

```
RandomForestRegressor(n_estimators=300, n_jobs=-1, random_state=42)
```

```
#Predict on the test set
test_predictions = rf.predict(df_test_encoded)
```

Drawing a Conclusion



Thank You